Chapter 3: Stacks

Technical / interview questions, exercises, multiple choice questions and answer key for Chapter 3 (Stacks) of Data Structures.

Programming Exercises

1. Implement a basic stack in C using an array.
2. Develop a function to push an element onto a stack.
3. Write a program to pop an element from the stack.
4. Implement a function to check if a stack is empty.
5. Develop a program to display the top element of a stack without remov ing it.
6. Write a C program to reverse a string using a stack.
7. Implement a function to check for balanced parentheses using a stack.
8. Develop a program to convert an infix expression to postfix using a stack.
9. Write a function to evaluate a postfix expression using a stack.
10. Implement a program to simulate the undo feature using a stack.
11. Create a stack using a linked list in C.
12. Write a function to push an element onto a linked list-based stack.
13. Develop a program to pop an element from a linked list-based stack.
14. Implement a function to check if a linked list-based stack is empty.
15. Write a program to convert a decimal number to binary using a linked list-based stack.
16. Develop a C program to implement a simple calculator using a stack.
17. Write a function to check for palindrome strings using a stack.
18. Implement a program to solve the Tower of Hanoi problem using three stacks.
19. Develop a C program to evaluate a prefix expression using a stack.
20. Write a function to implement a queue using two stacks.
21. Create a program to evaluate a mathematical expression with multiple operators using a stack.
22. Write a function to convert an infix expression to prefix using a stack.
23. Develop a C program to evaluate a postfix expression with variables using a stack.
24. Implement a program to validate and evaluate a boolean expression using a stack.
25. Write a function to convert a prefix expression to infix using a stack.
26. Create a program to implement the N-Queens problem using a stack for backtracking.
27. Implement a function to solve the rat in a maze problem using a stack for backtracking.
28. Write a C program to find all possible paths in a maze using a stack for backtracking.
29. Develop a function to solve the Sudoku puzzle using a stack for back tracking.
30. Implement a program to generate all possible combinations of a set using a stack for backtracking.
31. Write a C program to reverse a linked list using a stack.
32. Implement a function to reverse the order of words in a sentence using a stack.
33. Develop a program to reverse the order of elements in an array using a stack.
34. Create a function to reverse the order of characters in a string using a stack.
35. Write a program to reverse the order of paragraphs in a text file using a stack

Technical / Interview Questions

1. What is a stack?
2. Explain the fundamental concept of a stack and its LIFO (Last In, First Out) property.
3. How is a stack different from a queue?
4. Compare and contrast stacks and queues in terms of their operations and use cases.
5. What are the primary operations performed on a stack?
6. Discuss the basic operations of a stack, including push, pop, and peek.
7. Why is a stack considered a dynamic data structure?
8. Explain why stacks are considered dynamic and how their size can change during execution.
9. What is the significance of the top pointer in a stack?
10. Discuss the role of the top pointer in a stack and its importance in stack operations.
11. Implementing Stacks Using Arrays:
12. How do you implement a stack using an array?
13. Describe the steps involved in implementing a stack using a fixed-size array.
14. Explain the advantages and limitations of using an array-based stack.
15. Discuss the pros and cons of implementing a stack using an array.
16. What is the time complexity of push and pop operations in an array-based stack?
17. Analyze the time complexity of basic stack operations using an array.
18. Implementing Stacks Using Linked Lists:
19. How do you implement a stack using a linked list?
20. Explain the process of implementing a stack using a singly linked list.
21. Compare the advantages of a linked list-based stack over an array-based stack.
22. Discuss the benefits of using a linked list for implementing a stack.
23. What is the time complexity of push and pop operations in a linked list-based stack?
24. Analyze the time complexity of basic stack operations using a linked list.
25. Applications of Stacks:
26. Explain the role of stacks in function calls and recursion.
27. Discuss how stacks are used in managing function calls and recursion.
28. How can a stack be used to check for balanced parentheses in an expres sion?
29. Describe the algorithm for using a stack to validate parentheses in an expression.
30. Discuss the application of stacks in backtracking algorithms.
31. Explain how stacks are utilized in backtracking scenarios.
32. Application of Stacks for Expression Evaluation:
33. How does a stack play a crucial role in evaluating arithmetic expressions?
34. Discuss the process of using a stack to evaluate mathematical expressions.
35. Explain the steps involved in converting an infix expression to postfix using a stack.
36. Provide a step-by-step explanation of the infix to postfix conversion al gorithm.
37. How can a stack be employed to evaluate postfix expressions efficiently?
38. Discuss the algorithm for using a stack to evaluate postfix expressions.
39. Application of Stacks for Backtracking:
40. Describe how a stack is used in backtracking algorithms for problem-solv ing.
41. Discuss the role of stacks in scenarios requiring backtracking.

42. Explain the implementation of the N-Queens problem using a stack for backtracking.
43. Provide an overview of using a stack to solve the N-Queens problem.
44. Application of Stacks for Reversing a List:
45. How can a stack be utilized to reverse the elements of a list efficiently?
46. Describe the algorithm for reversing a list using a stack.
47. Discuss the role of a stack in reversing the order of words in a sentence.
48. Explain how stacks can be applied to reverse the order of words in a sentence.

Multiple Choice Questions

1. What is the primary characteristic of a stack?

A. FIFO
B. LIFO
C. Random Access
D. Dynamic Size

2. Which of the following operations is NOT typically performed on a stack?

A. Push
B. Pop
C. Insert
D. Peek

3. Why is a stack considered a dynamic data structure?

A. It changes size during program execution
B. It has a fixed size
C. It is static
D. It allows random access

4. What is the major disadvantage of an array-based stack?

A. Limited capacity
B. Dynamic size
C. Fast access time
D. Efficient memory usage

5. What is the time complexity of the pop operation in an array-based stack?

A. O(1)
B. O(n)
C. O(log n)
D. $O(n^2)$

6. How is a stack represented in memory when implemented using an array?

A. Elements scattered across memory
B. Contiguous block of memory
C. Linked nodes
D. Tree structure

7. What is a key advantage of a linked list-based stack over an array-based stack?

A. Fixed size
B. Limited flexibility
C. Dynamic size
D. Faster access time

8. What is the time complexity of the pop operation in a linked list-based stack?

A. O(1)
B. O(n)
C. O(log n)
D. $O(n^2)$

9. How is a stack represented in memory when implemented using a linked list?

A. Elements scattered across memory
B. Contiguous block of memory
C. Linked nodes
D. Tree structure

10. In which scenario would a stack be used for backtracking?

A. Searching
B. Sorting
C. Decision-making problems
D. Memory allocation

11. How are stacks involved in function calls and recursion?

A. To store local variables
B. To manage function calls
C. To handle recursion
D. All of the above

12. What is the role of a stack in evaluating postfix expressions?

A. To store operators
B. To store operands
C. To store both operators and operands
D. To store parentheses

13. How does a stack assist in converting an infix expression to postfix?

A. To reverse the expression
B. To store operators temporarily
C. To store operands temporarily
D. To validate parentheses

14. What is a common application of a stack in backtracking algorithms?

A. Sorting
B. Searching
C. Navigating a maze
D. Data compression

15. How does a stack contribute to solving the N-Queens problem?

A. To store chessboard configurations
B. To manage queen placements
C. To validate solutions
D. All of the above

16. How does a stack reverse the order of elements in a list?

A. By swapping elements
B. By reversing pointers
C. By using a temporary array
D. By utilizing recursion

17. In reversing the order of words in a sentence, what does a stack store?

A. Characters
B. Words
C. Sentences
D. Punctuation marks

Answer Key

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
1	B	2	C	3	A	4	A	5	A
6	B	7	C	8	A	9	C	10	C
11	D	12	B	13	B	14	C	15	B
16	B	17	B						